Guia da prova

Organização de Recursos — o projeto explicado de forma simples, com tudo o que o professor pediu, o que fizemos e como provar.

Disciplina	Qualidade de Software 2026.2 — Senac (prof. Afonso Lelis)
Turma	STADSCAS4NB (quinta-feira)
Integrantes	Guilherme Viana Lopes · Rafael Souza Silva · Gustavo Santana
Versão avaliada	tag `v1.0.0-rc3` (commit `e4e92c0`)
Repositório	github.com/guilhermevianacloud/organizacao-recursos
Entrega	12/11/2026 · **Prova oral: 19/11/2026**

Como usar este guia

Leia na ordem: em 1 minuto você entende o sistema; depois vê o que foi pedido, como o projeto é montado, os testes, os defeitos e os resultados. A última parte é o treino da prova oral.
Caixas **roxas** = **o que falar na prova**. Caixas **amarelas** = limite ou cuidado. Todo número vem de um documento do repositório, citado ao lado.

Sumário

1. O projeto em 1 minuto
2. O que o professor pediu × o que fizemos
3. Desmembrando o projeto: linguagem, escopo, estrutura e arquitetura
4. Os testes que fizemos
5. Defeitos encontrados e corrigidos
6. Resultados, riscos e decisão de qualidade
7. Quem fez o quê
8. Preparação para a prova oral
9. Glossário rápido

Documento de estudo do grupo, montado a partir dos documentos da pasta `docs/` e do `README.md`. Se algum número divergir, vale o documento de origem citado.

1 O projeto em 1 minuto

O problema

Reservas em planilha ou e-mail geram a mesma sala, material ou professor alocados duas vezes no mesmo horário, reservas em sala interditada, uso de equipamento caro sem aprovação e nenhum histórico de quem mudou o quê.

O sistema

Um sistema web para reservar **salas, professores e materiais sem conflito de horário**. Cada reserva aceita é garantida, mesmo quando duas pessoas pedem o mesmo recurso no mesmo instante, e toda mudança fica registrada.

O que é avaliado

Não basta funcionar: é preciso **provar com evidência** (testes, CI, cobertura, análise estática, carga, rastreabilidade e defeitos registrados).

Os três perfis de usuário

Perfil	Quem é	O que pode fazer no sistema
Solicitante	professor ou coordenador	pesquisar recursos livres, reservar, alterar e cancelar **as próprias** reservas
Responsável	coordenador de laboratórios	aprovar ou rejeitar recursos restritos, registrar retirada, devolução e não comparecimento, ver histórico e relatório
Administrador	secretaria acadêmica	cadastrar recursos, bloquear para manutenção, ver histórico e relatório

Uma reserva do começo ao fim

1. Login recebe um token (JWT) → **2. Pesquisa** filtra por tipo, capacidade, local, competência e horário livre → **3. Reserva** nasce SOLICITADA; se o horário já está ocupado → **409** → **4. Aprovação** recurso comum: automática · restrito: o Responsável aprova → **5. Uso** retirada → EM_USO · devolução → CONCLUÍDA

Cada passo gera um registro de auditoria imutável e uma notificação (simulada) depois que a mudança é gravada.

Os 4 limitadores de nota (o que derruba a nota, mesmo com o resto bom)

1) aplicação que não executa · 2) pipeline de CI ausente · 3) falha de autorização · 4) possibilidade de dupla reserva.
Por isso o projeto atacou esses quatro **primeiro**, e cada um tem teste que prova (seção 2).

A frase que resume a disciplina

"Uma afirmação de qualidade só vale se aponta para uma evidência e diz o que essa evidência **não** prova."

2 O que o professor pediu × o que fizemos

Fonte: especificação do projeto (site do prof. Afonso Lelis), `docs/prd.md` e `docs/status-requisitos.md`. Todos os itens abaixo têm teste automatizado verde no CI.

2.1 Os 14 requisitos funcionais obrigatórios

ID	O que foi pedido	O que fizemos / teste que prova	
RF-01	Autenticação e autorização por perfil	Login com JWT; sem token → 401; perfil errado → 403. `MatrizDeAutorizacaoIT`, `ApiDeReservasIT`	feito
RF-02	Cadastro e consulta de salas, professores e materiais	Só o Administrador cadastra; dado inválido → 400. Matriz (REC-01 a 04)	feito
RF-03	Pesquisa por tipo, capacidade, localização, competência e disponibilidade	5 filtros combináveis. `PesquisaDeRecursosIT`	feito
RF-04	Criar, alterar e cancelar reservas	Cancelar muda o estado, nunca apaga. Matriz (RES-03 a 07), `ServicoDeReservasIT`	feito
RF-05	Sobreposição de horários, inclusive na agenda do professor	Professor é um "recurso"; mesma regra vale para os 3 tipos. `IntervaloTest`, `ServicoDeReservasIT`	feito
RF-06	Proteção contra dupla reserva simultânea	20 pedidos ao mesmo tempo → 1 aceito. `DuplaReservaConcorrenciaIT`	feito
RF-07	Aprovação obrigatória de recursos restritos	Restrito fica SOLICITADA até o Responsável aprovar. BDD + matriz (RES-08)	feito
RF-08	Bloqueio de recursos em manutenção	Pedido no período da manutenção é recusado; fronteiras testadas ao segundo	feito
RF-09	Retirada e devolução de materiais	Retirada → EM_USO; devolução → CONCLUÍDA; ambas auditadas	feito
RF-10	Histórico auditável	Registro imutável na mesma transação. `AuditoriaIT`	feito
RF-11	Notificação simulada ou API externa	Provedor HTTP testado com WireMock; falha não desfaz a reserva. `NotificadorHttpIT`	feito
RF-12	Relatório de utilização, carga horária e conflitos evitados	Valores conferidos à mão no teste. `RelatorioDeUtilizacaoIT`	feito
RF-13	Interface responsiva e mensagens compreensíveis	Interface web; sem rolagem lateral em 360 px; erros em português. `InterfaceE2EIT`	feito
RF-14	Documentação da API ou dos fluxos	OpenAPI/Swagger em `/swagger-ui.html` + README. `DocumentacaoDaApiIT`	feito

2.2 As regras críticas

Regra	O que diz	Teste que prova
RN-01	O término deve ser depois do início	`IntervaloTest`, `ApiDeReservasIT`
RN-02	Reservas ativas do mesmo recurso não se sobrepõem (sala, material, professor)	`IntervaloTest`, `ServicoDeReservasIT`
RN-03	Dois pedidos simultâneos → uma única reserva aceita	`DuplaReservaConcorrenciaIT`, `ProtecaoNoBancoIT`
RN-04	Recurso em manutenção não pode ser reservado	`ServicoDeReservasIT`, BDD

RN-05 / 06	Só o Responsável aprova restrito; quem pediu não aprova a própria	BDD, matriz, `ServicoDeReservasTest`
RN-07	Só as transições de estado permitidas	`EstadoReservaTest` (49 pares), BDD
RN-08	Reserva iniciada não pode ser apagada	`ReservaTest` , `AuditoriaIT`
RN-09	Toda mudança de estado gera auditoria	`AuditoriaIT`
RN-10	Solicitante só mexe nas próprias reservas	`MatrizDeAutorizacaoIT` (IDOR)

2.3 Estratégia obrigatória de qualidade

Pedido do professor	Como cumprimos
Unitários JUnit 5, parametrizados, integração, API caixa-preta e end-to-end	98 testes de unidade + 187 de integração/API/BDD/E2E (seção 4)
Banco em container (Testcontainers) e WireMock	PostgreSQL 17 real nos testes; WireMock no provedor de notificação
Teste automatizado de concorrência	`DuplaReservaConcorrenciaIT` : 20 pedidos simultâneos, 5 repetições
TDD ou BDD com evidência	Os dois: commits `test(red)` → `feat(green)` e 18 cenários Cucumber
GitHub Actions em PR, JaCoCo e SonarCloud	Workflow `qualidade` ; gate 80/70; Quality Gate aprovado
JMeter, autorização, validação e erro seguro	Rodada de carga; matriz com 88 células; 400 com mensagem; erro sem detalhe interno
Mock só em unidade isolada	Mockito só em `ServicoDeReservasTest` ; o resto usa banco real

2.4 Metas mensuráveis

Cobertura de linhas (meta 80%)

98,0%

583 de 595 · atingida

Cobertura de ramos (meta 70%)

95,0%

134 de 141 · atingida

Críticos na RTM (meta 100%)

9 de 9

com teste e mutação · atingida

Bugs/vulnerabilidades críticas (meta 0)

0

Quality Gate aprovado, segurança A · atingida

2.5 Os 12 entregáveis da especificação

Entregável pedido	Onde está no repositório
Repositório com histórico e revisão por pares	GitHub, pull requests #1 a #26; o Rafael aprovou o PR #25
Aplicação executável e instruções de instalação	`README.md` (testado em clone limpo: `docs/entrega/checklist.md`)
README e documentação da API/fluxos	`README.md` + Swagger (`/swagger-ui.html`)
RTM ligando requisito, risco, teste e evidência	`docs/rtm.md` (e `RTM.md` na raiz apontando para ela)
Diagramas de contexto, componentes e sequências	`docs/arquitetura.md` (C4 níveis 1–3, modelo de dados, 2 sequências)
Plano, suíte automatizada e relatório de testes	`docs/plano-de-teste.md` , `src/test/` , `docs/relatorio-qualidade.md`
Pipeline de integração contínua	`.github/workflows/ci.yml`
Relatórios do SonarCloud e do JaCoCo	SonarCloud (branch `main`) + artifact `relatorio-de-cobertura` do CI

Plano e resultado do teste de carga	`perf/reservas.jmx` , `docs/performance/resultado-01.md` , `.jtl` versionado
Registro de defeitos priorizados e corrigidos	`docs/defeitos/` (DEF-001 a 006) + issues com rótulo `bug`
Relatório final de qualidade	`docs/relatorio-qualidade.md` + `docs/decisao-qualidade.md`
Apresentação e demonstração	Prova oral em 19/11; roteiro em `docs/prova-oral/ensaio.md`

2.6 Aula por aula: o que cada aula pediu e o que entrou no projeto

Aula	Tema	O que fizemos no projeto (arquivo)
01	Introdução à qualidade	Vocabulário usado em tudo: qualidade, verificação × validação, erro → defeito → falha, ISO 25010
02	ATAM e casos de teste	Plano de teste no formato ISO 29119-3 (`docs/plano-de-teste.md`); cenários de qualidade em 6 partes
03	Caixa branca × preta; requisitos com IA	PRD com personas, escopos, 14 RF, 14 RNF com número e regras (`docs/prd.md`)
04	Arquitetura e ATAM com IA	C4, utility tree, riscos, trade-offs e ADRs (`docs/arquitetura.md` , `docs/adr/`)
05	Gestão de erros; TDD fase red	Inventário de testes por regra (`docs/testes/plano-tdd.md`); registro de defeitos
06	Testes em CI; TDD green; 1º ATAM	GitHub Actions com quebra intencional e reversão (PR #1); ciclo ATAM 01
07	JUnit 5 + JaCoCo + SonarCloud	Gate 80/70 no `verify` ; SonarCloud no workflow (DEF-004)
08	WireMock e APIs seguras	Notificação por porta + adaptador; matriz de falhas INT-01 a 06 (ADR-005)
09	TDD e BDD	Aprovação e bloqueio em Cucumber (`aprovacao-bloqueio.feature`); evidência red/green
10	Carga e concorrência	20 pedidos simultâneos; JMeter (`docs/performance/`); DEF-001, 002 e 003
11	Auditoria e relatórios	Auditoria na mesma transação; relatório de utilização; status dos requisitos
12	Autorização e RTM	Matriz de autorização lida pelo teste; RTM fechada; mutação manual (11 de 11)
13	Preparação para a prova oral	Relatório de qualidade; banco de 15 perguntas (`docs/prova-oral/`)
14	Entrega final	Tag `v1.0.0-rc3` , índice de evidências, checklist em clone limpo, decisão de qualidade

3 Desmembrando o projeto

3.1 Linguagem e ferramentas (a "stack")

Peça	O que é, em uma frase	Por que usamos
Java 21	A linguagem de programação do sistema	Exigida pela especificação
Spring Boot 3.5.16	Framework que já traz servidor web, segurança e acesso a banco prontos	Exigido "Spring Boot 3.x" (ADR-001)
PostgreSQL 17	O banco de dados	Tem o recurso que impede a dupla reserva (exclusion constraint)
Flyway	Cria e atualiza as tabelas por scripts numerados (V1 a V4)	O banco fica igual em toda máquina
Docker	Roda o PostgreSQL num "container", sem instalar o banco	Aplicação e testes sobem o banco sozinhos
Maven Wrapper	Compila e roda os testes (`./mvnw`)	Não precisa instalar o Maven
Spring Security + JWT	Login que devolve um "crachá" digital (token) com validade de 60 min	Cada pedido mostra o token (ADR-003)
HTML + CSS + JavaScript	A interface, servida pela própria aplicação	Um único programa para subir (ADR-006)

Como rodar (o professor segue só o README)

Precisa de **JDK 21** e **Docker** ligado. Testes: `./mvnw -B verify` (no Windows, `.\mvnw.cmd -B verify`). Aplicação: `./mvnw spring-boot:run` e abrir `http://localhost:8080`. Usuários de demonstração: `solicitante@demo.invalid`, `responsavel@demo.invalid` e `admin@demo.invalid`; a senha de demonstração está no README.

3.2 Escopo: o que está dentro e o que ficou fora

Dentro (5 escopos)

E1 Identidade e acesso (login, perfis, posse)
E2 Recursos (cadastro e pesquisa)
E3 Reservas e agenda (sobreposição, dupla reserva)
E4 Aprovação, manutenção e materiais
E5 Auditoria, relatórios e notificações

Fora (decisão do grupo, no PRD)

Integração com o sistema acadêmico real e login institucional · envio real de e-mail/SMS (notificação simulada) · reservas recorrentes e lista de espera · aplicativo nativo (a web é responsiva) · cobrança ou multa

3.3 Estrutura de pastas

```
organizacao-recursos/
├── README.md, CHANGELOG.md, RTM.md      → como rodar, versões, atalho para a RTM
├── compose.yaml                         → PostgreSQL em container para rodar a aplicação
├── .github/workflows/ci.yml             → o pipeline de CI (workflow "qualidade")
├── perf/                                → plano do JMeter (reservas.jmx) e resultado (rodada-01.jtl)
├── docs/                                → toda a documentação (PRD, arquitetura, testes, defeitos, RTM...)
└── src/
    ├── main/java/br/senac/organizacaorecursos/
    │   ├── api/            → controllers REST (Autenticacao, Recurso, Reserva, Manutencao, Relatorio) e erros
    │   ├── seguranca/      → Spring Security, geração e leitura do token JWT
    │   ├── servico/        → regras de uso: ServicoDeReservas, Recursos, Manutencao, Relatorios
    │   ├── dominio/        → o "coração": Intervalo, EstadoReserva, Reserva, Recurso, Auditoria
    │   ├── persistencia/   → repositórios JPA (conversa com o banco)
    │   ├── notificacao/    → porta Notificador + adaptadores HTTP e simulado
    │   └── configuracao/   → relógio (Clock) e dados de demonstração
    ├── main/resources/     → db/migration (V1 a V4) e static (telas: HTML, CSS, JS)
    └── test/               → 21 arquivos Java de teste + o arquivo .feature do BDD
```

3.4 Arquitetura em camadas (de cima para baixo)

Navegador	telas (Agenda do dia, Nova reserva, Minhas reservas, Aprovações, Recursos, Manutenção, Relatório)
api + seguranca	recebe o pedido HTTP, confere o token e o perfil, valida a entrada, devolve erro em português (ProblemDetail)
servico	executa o caso de uso: reservar, aprovar, cancelar, retirar, devolver; grava a auditoria na mesma transação
dominio	regras puras: sobreposição de intervalos e transições de estado (as mais testadas)
persistencia / notificacao	banco PostgreSQL (com a trava e a constraint) e o envio de aviso para fora

C4 nível 1 — Contexto

Os 3 perfis usam o sistema; o sistema avisa um provedor de notificação externo (simulado) por HTTP.

C4 nível 2 — Contêineres

Só 2 peças: a aplicação Spring Boot (API + telas) e o PostgreSQL. O banco é o ponto único de verdade sobre conflitos.

3.5 Os dados (tabelas)

usuario (perfil, senha com hash BCrypt) · **recurso** (tipo SALA, MATERIAL ou PROFESSOR, capacidade, local, competências, restrito) · **reserva** (recurso, solicitante, início, término, estado) · **manutencao** · **auditoria_reserva** (quem, quando, estado anterior → novo, motivo) · **tentativa_recusada** (cada 409, para contar os "conflitos evitados" no relatório).

> **Ideia-chave: professor também é "recurso"**
>
> Sala, material e professor ficam na mesma tabela, com um campo `tipo`. Assim **uma só regra** de sobreposição protege os três, e a "agenda do professor" (RF-05) sai de graça.

3.6 Máquina de estados da reserva

SOLICITADA → APROVADA → EM_USO → CONCLUIDA

alternativos: REJEITADA, CANCELADA

NAO_COMPARECEU

Toda reserva nasce SOLICITADA. Recurso comum é aprovado automaticamente; restrito espera o Responsável. Estados finais não mudam mais. Só **SOLICITADA, APROVADA e EM_USO ocupam** o recurso (cancelada não bloqueia a sala). O teste `EstadoReservaTest` confere os **49 pares** possíveis (7 estados × 7).

3.7 Como a dupla reserva é impedida (a parte mais cobrada)

A e B pedem a sala 101 no mesmo instante → **Trava** `SELECT ... FOR UPDATE` no recurso: B espera A terminar → **A grava** reserva + auditoria; COMMIT → 201 → **B confere** vê o conflito → ROLLBACK → 409 e grava a tentativa recusada

Duas camadas (ADR-002): a **trava por recurso** faz os pedidos esperarem a vez (evita deadlock) e a **exclusion constraint** no PostgreSQL é a garantia final: o banco recusa duas linhas do mesmo recurso com horários que se cruzam, mesmo que alguém grave direto por SQL.

> **Na prova**
> "Verificar no código se a sala está livre não basta: entre verificar e gravar, outro pedido entra. `synchronized` também não basta, porque protege uma instância da aplicação, não várias. O banco é o único ponto que todos compartilham."

3.8 As 7 decisões de arquitetura (ADRs)

ADR	Decisão	Por quê / custo
001	Spring Boot 3.5.16	Especificação pede 3.x. Custo: a versão está fora do suporte gratuito (risco assumido)
002	Trava por recurso + exclusion constraint	Garante 1 reserva por horário. Custo: fila no pico (DEF-003)
003	Autenticação por JWT (60 min)	Sem sessão no servidor. Custo: token não pode ser cancelado antes de vencer
004	Intervalos semiabertos `[início, término)`	8h–10h e 10h–12h **não** conflitam (encostam, mas não se cruzam)
005	Notificação por porta + adaptador HTTP	Envio depois do commit, com timeout de 2 s: falha externa não desfaz reserva
006	Interface HTML/CSS/JS servida pelo Spring; E2E com Playwright	Um único programa para subir; teste da interface em navegador real
007	CSRF desligado	API sem cookie (token no cabeçalho). Custo: se um dia usar cookie, precisa religar

3.9 Avaliação da arquitetura (ATAM)

Ponto de sensibilidade

O operador `<` × `<=` na sobreposição decide sozinho se reservas encostadas conflitam (mutação M1 prova).

Ponto de trade-off

A trava melhora a **integridade** e piora a **latência** no pico (medido no DEF-003). Cache de disponibilidade: descartado, poderia mostrar livre uma sala já ocupada.

Riscos tratados

R-01 constraint contando canceladas (só estados que ocupam) · R-02 IDOR (posse no serviço) · R-03 notificação dentro da transação (envio depois do commit).

Não-risco

Queda do provedor de notificação não afeta reservas: o envio é fora da transação e com timeout.

Fontes: `docs/arquitetura.md` (seção 11) e `docs/avaliacao-arquitetural/ciclo-01.md`, que refez a análise sobre o código pronto.

4 Os testes que fizemos

Testes automatizados	Unidade	Integração, API, BDD, E2E	Pedidos na carga
285	**98**	**187**	**18.899**
0 falhas no CI	JUnit 5, AssertJ, Mockito	banco real (Testcontainers)	JMeter, 0 erro real

4.1 Tipos de teste: a pergunta que cada um responde

Tipo	Pergunta	Ferramenta	Exemplo no projeto
Unidade	A regra isolada está certa?	JUnit 5, AssertJ	`IntervaloTest` : sobreposição com limites −1 / 0 / +1 segundo
Parametrizado	Vale para a tabela inteira de casos?	`@ParameterizedTest`	`EstadoReservaTest` (49 pares); matriz de autorização célula a célula
Mock (unidade)	O serviço decide certo sem banco?	Mockito	`ServicoDeReservasTest` (ex.: quem pediu não aprova)
Integração	As peças funcionam juntas com o banco real?	Testcontainers	`AuditoriaIT` : rollback não deixa auditoria órfã
Concorrência	Resiste a pedidos simultâneos?	`ExecutorService` + `CountDownLatch`	`DuplaReservaConcorrenciaIT` : 20 → 1 aceito + 19 com 409, 5 vezes; também na agenda do professor
Banco	O banco barra sozinho?	SQL direto	`ProtecaoNoBancoIT` : a constraint recusa sobreposição mesmo sem a aplicação
API caixa-preta	O contrato HTTP está certo?	MockMvc	`RelatorioDeUtilizacaoIT` com valores calculados à mão; erro sem detalhe interno
Segurança	Cada perfil só faz o que pode?	MockMvc parametrizado	`MatrizDeAutorizacaoIT` : **88 células**, inclusive IDOR
Integração externa	E se o provedor falhar?	WireMock	`NotificadorHttpIT` : INT-01 a 06 (200, 404, 500, timeout, JSON ruim, segredo fora do log)
BDD	A regra de negócio escrita é a que roda?	Cucumber	`aprovacao-bloqueio.feature` : 18 cenários Dado/Quando/Então
Ponta a ponta	O fluxo funciona pela tela?	Playwright (Chromium)	`InterfaceE2EIT` : 8 testes (login → reservar → aprovar, 360 px, campos vazios)
Documentação	A API está documentada e pública?	MockMvc	`DocumentacaoDaApiIT` : OpenAPI e Swagger sem login
Carga	O desempenho atende à meta?	JMeter 5.6.3	50 usuários, 70 s (4.6)
Mutação manual	O teste protege mesmo a regra?	script + `git restore`	11 de 11 quebras detectadas (4.7)
Exploratório	O que o uso real revela?	navegador	O Guilherme achou o DEF-005 usando o sistema

Mock × Testcontainers (pergunta clássica)

Mock só em unidade isolada, como manda a especificação: é rápido e alcança caminhos difíceis. Banco, transação e constraint são testados com **PostgreSQL real** (Testcontainers). **Não usamos H2**: ele não tem `tstzrange` nem exclusion constraint, então o teste passaria num banco diferente do real.

4.2 TDD: o teste antes do código

Ciclo **red** (o teste falha pela razão certa) → **green** (código mínimo até passar). Os commits `test(red)` e `feat(green)` mostram isso no histórico. Fonte: `docs/testes/evidencia-tdd.md`.

Ciclo	Commit red	Saída no red	Commit green
RN-01 e RN-02 (`IntervaloTest`)	`57ee0cf`	13 testes, 8 falhas	`b18d2f1`
RN-07 transições, RN-08 (`EstadoReservaTest` , `ReservaTest`)	`a711262`	85 testes, 55 falhas	`06cd9c3`
RN-02 no banco, RN-04 manutenção	`cbd0394`	17 testes, 8 falhas	`3151b6d`
RN-03 20 pedidos simultâneos	`f68d7f4`	"aceitos: esperado 1, foi 10"	`152c20e`
Segurança e API (matriz)	`407bd67`	86 testes, 67 falhas	`aa60caa`

Dizer com honestidade

O prompt da Aula 09 pede um teste por vez; nós escrevemos vários testes da mesma regra num commit vermelho e conferimos a falha de cada um. Não houve commit `refactor` porque o código verde não teve duplicação. Está declarado no próprio documento.

4.3 BDD com Cucumber

Arquivo `src/test/resources/features/aprovacao-bloqueio.feature` : aprovação de restrito, bloqueio por manutenção (com tabela de fronteiras) e transições, inclusive REJEITADA e NAO_COMPARECEU. São **18 cenários**. Eles foram escritos depois das regras feitas por TDD e passaram na primeira execução, e isso está declarado.

4.4 Matriz de autorização

`docs/seguranca/matriz-autorizacao.md` é uma tabela operação × perfil (anônimo, Solicitante, Responsável, Administrador) com o código esperado (200, 201, 401, 403). O teste **lê a própria tabela** e executa as 88 células com token real. Mudou a matriz, mudou o teste.

401 Unauthorized

Não está logado (sem token ou token adulterado/vencido).

403 Forbidden

Está logado, mas o perfil não pode (ex.: Solicitante aprovando).

IDOR = trocar o id na URL para mexer na reserva de outro. Células RES-06 e RES-07: o Solicitante tenta cancelar ou alterar a reserva de outro → **403**.

4.5 CI, cobertura e análise estática

- **GitHub Actions** (workflow `qualidade` , job `test`): a cada pull request e a cada push na `main` , roda `./mvnw -B verify` num servidor limpo. Os relatórios ficam guardados 90 dias.
- **Por que `verify` e não `test` :** o `verify` roda também os testes de integração e o **gate** do JaCoCo.
- **Branch protection:** a `main` só aceita merge com o CI verde. Prova da Aula 06 (PR #1): a execução 41207912465 ficou **vermelha de propósito** e voltou a verde com `git revert` .
- **JaCoCo:** o build **falha** abaixo de 80% de linhas ou 70% de ramos (provado subindo a meta para 99%).

- **SonarCloud:** a 1ª análise achou 12 apontamentos (DEF-004); 11 foram corrigidos e 1 (S4502, CSRF) foi justificado no ADR-007 e aceito. Resultado: Quality Gate aprovado, nota de segurança A.

> **97,4% no Sonar × 98,0%/95,0% no JaCoCo**
> É o mesmo relatório. O Sonar junta linhas e ramos num número só: (583 + 134) ÷ (595 + 141) = 717 ÷ 736 = 97,4%.

4.6 Teste de carga com JMeter

A meta foi escrita **antes** de medir (`docs/performance/rnf.md`). Carga: 50 usuários, 70 s, 70% consultas e 30% criações, **todos disputando a mesma sala e o mesmo horário** (pior caso de propósito).

Operação	Pedidos	p95	Meta	Respostas	Erro real
Consulta de disponibilidade	13.197	**332 ms**	≤ 300 ms não atende	200 = 13.197	0%
Criação de reserva	5.652	**506 ms**	≤ 300 ms não atende	201 = 1 · 409 = 5.651	0%
Integridade (RN-03)	1 reserva aceita em 5.652 tentativas; 5.651 tentativas recusadas gravadas no banco atende				

> **p95 em português**
> 95% dos pedidos foram mais rápidos que esse valor. É mais honesto que a média, que esconde os lentos.

> **Por que 409 não é erro**
> Na disputa pela mesma sala, recusar com 409 é o comportamento **correto**. Erro real seria 500 ou falha de asserção: zero.

> **Limites (o que o p95 NÃO permite afirmar)**
> Foi medido numa máquina local, com o JMeter na mesma máquina, numa única rodada e com um único usuário no CSV. **Não diz nada sobre produção.** A causa provável é a fila na trava do recurso, mas ainda é hipótese (DEF-003).

4.7 Mutação manual: "o teste protege mesmo?"

Quebramos cada regra crítica **de propósito**, rodamos os testes e desfizemos com `git restore` . Se o teste continuasse verde, seria uma lacuna. Para reproduzir: `sh scripts/mutacao-manual.sh M5` .

#	Regra	Quebra feita	Teste que pegou
M1	Sobreposição	`<` trocado por `<=`	`IntervaloTest`
M2	Dupla reserva	tirou a trava e a constraint	`DuplaReservaConcorrenciaIT` , `ProtecaoNoBancoIT`
M3	Aprovação de restrito	todo recurso aprovado na criação	`ReservaTest` , BDD
M4	Manutenção	verificação removida	`ServicoDeReservasIT` , BDD
M5	Posse (IDOR)	conferência do dono removida	Matriz (RES-06, RES-07)
M6	Perfil que aprova	`@PreAuthorize` removido	Matriz (RES-08)
M7	Auditoria	gravação removida	`AuditoriaIT`
M8	Estados	SOLICITADA → EM_USO liberado	`EstadoReservaTest` , `ReservaTest`
M9	Término após início	duração zero aceita	`IntervaloTest`
M10	Reserva iniciada	pode ser apagada	`ReservaTest`

M11	Quem pediu não aprova	conferência removida	`ServicoDeReservasTest`

Resultado: 11 de 11 detectadas

As 9 regras críticas da RTM têm teste que pega a quebra (100%).

Na prova: "cobertura de 98% prova o quê?"

Prova que o código **rodou** nos testes, não que o resultado foi **verificado**: um teste sem asserção também dá cobertura. Por isso fizemos a mutação manual.

5 Defeitos encontrados e corrigidos

Cada defeito tem um arquivo em `docs/defeitos/` e uma issue no GitHub. Desde o DEF-005, a correção segue `test(regressao)` (teste que reproduz o defeito) → `fix(DEF-xxx)`.

ID	O que aconteceu	Severidade	Como achamos e corrigimos	Situação
DEF-001	20 pedidos simultâneos geraram várias reservas (10, 7, 6, 4, 6)	Crítica / P0	Achado pelo teste de concorrência; corrigido com exclusion constraint + trava	corrigido
DEF-002	Com só a constraint, às vezes dava deadlock (erro 500)	Alta / P1	Mesmo teste ("exceções inesperadas"); corrigido com a trava por recurso	corrigido
DEF-003	p95 acima da meta sob carga (332 e 506 ms)	Média / P2	Achado pelo JMeter; causa ainda é hipótese	risco aceito
DEF-004	12 apontamentos da 1ª análise do SonarCloud	Alta / P1	11 corrigidos; S4502 justificado (ADR-007) e aceito	corrigido
DEF-005	Data vazia derrubava a agenda e o relatório ("Invalid time value")	Média / P1	Achado pelo Guilherme usando o sistema; 3 testes de interface	corrigido
DEF-006	Resposta atrasada da agenda apagava a tela aberta depois	Baixa / P2	Achado pelo teste E2E (falha intermitente); resposta retida de propósito no teste	corrigido

Severidade × prioridade

Severidade = tamanho do dano. **Prioridade** = urgência de corrigir. São independentes.

Erro → defeito → falha

Engano humano → problema no código → comportamento errado que alguém vê.

Na prova: "o DEF-002 aparecia às vezes. Vocês rodaram de novo até passar?"

Não. Teste intermitente é defeito a investigar, não "instabilidade". Era deadlock entre pedidos simultâneos; a trava por recurso faz os pedidos do mesmo recurso esperarem a vez. Depois disso, ele ficou 3 rodadas sem "exceções inesperadas", mas 3 rodadas não provam que nunca mais vai acontecer.

Também corrigimos problemas **dos testes** (não do produto): o teste de concorrência não limpava o banco; a 1ª tentativa da M2 foi inválida sem `clean`; testes de interface agiam antes de a tela carregar.

6 Resultados, riscos e decisão

Decisão de qualidade (2026-11-11): liberar com ressalvas a v1.0.0-rc3 para avaliação acadêmica

Todos os 14 RF com teste; os 4 limitadores tratados; metas de cobertura, RTM e análise estática atingidas. **Não** afirma prontidão para produção: tudo foi medido em ambiente local e no CI.

Risco aceito	Impacto	Por que aceitamos
DEF-003: p95 acima da meta	até ~0,5 s em 5% das criações, no pior caso	sem perda de dados nem erro; a meta foi proposta pelo grupo
CSRF desligado (S4502, ADR-007)	exposição se a credencial for para cookie	token vai no cabeçalho, não em cookie
Token JWT não revogável (ADR-003)	usuário desligado acessa por até 60 min	validade curta; revogar exigiria lista de bloqueio

Spring Boot 3.5 sem suporte gratuito (ADR-001)	falha nova sem correção pública	a especificação exige 3.x

Riscos assumidos por Guilherme Viana Lopes e Gustavo Santana em 2026-11-11 (`docs/decisao-qualidade.md`). Evidência da versão: CI 43089789523 no commit da tag (98 + 187 testes, 0 falhas).

7 Quem fez o quê

O código, os testes e a documentação foram escritos com um agente de IA (Claude), seguindo os prompts das aulas, como declarado no README. O grupo usou uma única conta do GitHub; em 2026-11-11 o histórico foi reescrito para registrar autor e coautores de cada commit, sem mudar nenhum arquivo. Divisão informada pelo grupo (`docs/decisao-qualidade.md`):

Assunto	Commits	Participantes
Esqueleto do projeto e build dos testes de integração	2	Guilherme, Rafael, Gustavo
Pipeline de CI (quebra intencional e reversão)	4	Guilherme
Requisitos, arquitetura, plano de teste e RTM	1	Guilherme, Gustavo
Regras de reserva com TDD (RN-01, 02, 04, 07, 08)	6	Guilherme, Rafael, Gustavo
Concorrência e trava no banco (RN-03, DEF-001, DEF-002)	3	Gustavo
Login, autorização, API e auditoria	5	Guilherme
Pesquisa, relatório e notificação com WireMock	4	Rafael, Gustavo
Cenários BDD com Cucumber	1	Guilherme, Rafael
Interface web e testes com Playwright (DEF-005, DEF-006)	5	Guilherme, Rafael, Gustavo
Cobertura (JaCoCo), SonarCloud e correções	4	Guilherme, Gustavo
Desempenho com JMeter e mutação manual	4	Guilherme, Gustavo
Documentação e entrega	11	Guilherme, Rafael, Gustavo
Identidade visual da interface (CSS)	4	Guilherme

> **Atenção para a prova**
> O professor avalia o **domínio de autoria**: cada integrante precisa explicar **todas** as partes, inclusive as que não fez. Por isso o banco de perguntas distribui as perguntas para cada pessoa responder sobre partes que não escreveu.

8 Preparação para a prova oral (19/11)

8.1 As 15 perguntas da banca, com a ideia da resposta

Distribuição de `docs/prova-oral/perguntas.md`. Responda **sem ler** e sempre cite a evidência (arquivo, teste ou número).

#	Pergunta	Ideia da resposta forte	Quem
1	Qual mecanismo impede a dupla reserva e qual teste prova?	Trava por recurso + exclusion constraint (ADR-002); `DuplaReservaConcorrenciaIT` 20 → 1, e `ProtecaoNoBancoIT`	Rafael
2	Por que mock num ponto e Testcontainers em outro?	Regra da especificação: mock só em unidade isolada; banco e transação com PostgreSQL real; H2 não tem a constraint	Gustavo
3	O que o p95 permite e não permite afirmar?	332/506 ms nesta máquina, neste cenário de pior caso; nada sobre produção	Guilherme
4	Qual teste falha se a aprovação de restrito for removida?	M3 e M6: `ReservaTest`, BDD e células RES-08 da matriz	Rafael
5	Por que 8h–10h não conflita com 10h–12h?	Intervalo semiaberto `[início, término)`, ADR-004; `IntervaloTest` (RN02-01, RN02-04)	Gustavo
6	O DEF-002 apareceu "às vezes". Rodaram até passar?	Não: era deadlock; corrigido com a trava; o teste não foi alterado	Guilherme
7	Como sabem que a auditoria está na mesma transação?	`AuditoriaIT.rn09_03_rollbackNaoDeixaAuditoriaOrfa`: força o rollback e confere que não sobra auditoria	Rafael
8	Cobertura de 98%: o que prova e o que não prova?	Que o código rodou, não que foi verificado; lacunas em `lacunas-de-cobertura.md`; mutação 11 de 11	Gustavo
9	Solicitante cancela a reserva de outro trocando o id?	Não: 403 (RES-06/07); autorização ≠ autenticação; mutação M5 prova o teste	Guilherme
10	De onde saíram as 3,5 h da Sala A no relatório?	Do cálculo à mão no comentário de `RelatorioDeUtilizacaoIT`, não do resultado da API	Rafael
11	Se o provedor de notificação cair, a reserva some?	Não: envio depois do commit, timeout 2 s (ADR-005); `NotificacaoAposCommitIT`, INT-03/04	Gustavo
12	Por que o CI roda `verify`? E se um teste falhar no PR?	`verify` inclui integração e o gate; branch protection bloqueia o merge; execução vermelha do PR #1	Guilherme
13	Diferença entre 401 e 403?	401 sem login/token inválido (`tokenAdulteradoResponde401`); 403 logado sem permissão	Rafael
14	O erro de validação vaza detalhe interno?	Não: `TratamentoDeErros` devolve ProblemDetail em português com `codigo`; `ApiDeReservasIT.semDetalheInterno`	Gustavo
15	Qual decisão tem mais trade-offs e qual o custo?	ADR-002: integridade × latência; o custo medido é a fila no pico (DEF-003)	Guilherme

8.2 O que o professor costuma pedir "na mesa" e como responder

Ele pede...	Você mostra...
O commit vermelho de um TDD	`docs/testes/evidencia-tdd.md` → ex.: `f68d7f4` (aceitos: esperado 1, foi 10)

Quebrar a regra de manutenção e ver qual teste falha	`sh scripts/mutacao-manual.sh M4` → `ServicoDeReservasIT.EmManutencao` e a feature
Uma linha da RTM até o teste	`docs/rtm.md` → arquivo do teste → execução do CI
Remover a proteção de concorrência	Mutação M2 → `DuplaReservaConcorrenciaIT` fica vermelho
De onde saiu o p95; se o 409 contou como erro	`perf/resultados/rodada-01.jtl` ; 409 não é erro (recusa correta)
A auditoria de uma reserva cancelada	Histórico pela API ou pela tela (Responsável/Administrador)
Refazer à mão uma linha do relatório	Comentário de cálculo em `RelatorioDeUtilizacaoIT`
Solicitante mexendo na reserva de outro	Requisição → 403; célula RES-06 da matriz
Provocar erro de validação e ler a resposta	Término antes do início → 400 com mensagem em português, sem stack trace
Um número do relatório e a fonte	`docs/relatorio-qualidade.md` (cada número tem a fonte ao lado)
Clonar e seguir o README	`docs/entrega/checklist.md` (clone limpo repetido em 2026-11-09)
A execução do CI ligada à tag	`docs/evidencias/README.md` → execução 43089789523 no `e4e92c0`

8.3 Roteiro da apresentação (8 minutos)

1. Problema dupla alocação e limitadores → **2. Arquitetura** C4, recurso único, proteção no banco → **3. Demonstração** login, reservar, 409, aprovação, relatório → **4. Testes e CI** TDD, matriz, BDD, E2E, gate → **5. RTM** 9 de 9, mutação 11 de 11, defeitos → **6. Limites** DEF-003, S4502, ambiente local

> **Falta fazer antes da prova**
> O ensaio com os três respondendo sem ler, com nota de 0 a 2 em correção técnica, evidência citada e domínio de autoria. Registrar as notas reais em `docs/prova-oral/ensaio.md` (ainda não realizado).

9 Glossário rápido

ADR	Registro curto de uma decisão: contexto, alternativas, decisão, consequências
ATAM	Método para avaliar a arquitetura: riscos, não-riscos, pontos de sensibilidade e de trade-off
C4	Diagramas em 4 níveis de zoom: contexto, contêineres, componentes, código
CI	Integração contínua: os testes rodam sozinhos a cada mudança enviada
Exclusion constraint	Regra do PostgreSQL que recusa duas linhas com intervalos que se cruzam
Flaky	Teste intermitente; passar no "rodar de novo" não resolve
IDOR	Acessar o dado de outro trocando o id na URL
JWT	Token assinado devolvido no login e enviado em cada pedido
Oráculo	O que diz se o resultado do teste está certo ("não deu exceção" não basta)
p95	95% dos pedidos foram mais rápidos que esse tempo

ProblemDetail	Formato padrão de resposta de erro (com `codigo` estável, ex.: `CONFLITO_DE_HORARIO`)
Quality Gate	Conjunto de condições do SonarCloud que aprova ou reprova o código
RTM	Matriz de rastreabilidade: requisito → risco → teste → evidência
Testcontainers	Sobe um PostgreSQL real em Docker durante o teste
WireMock	Servidor HTTP falso para simular a API externa (inclusive falhas)
Valor limite	Testar −1 / 0 / +1 em volta da fronteira (ex.: 1 segundo antes, no início, 1 segundo depois)